The Complete LeetCode Mastery Guide

From "I know the basics" to "I can solve anything" — no AI shortcuts, just real understanding

---

0. The Core Philosophy (read this first)

You already know basic DSA. The gap between that and "mastering LeetCode" is **pattern recognition** — recognizing which of ~15 underlying techniques a problem is really testing, even when it's disguised in a weird story.

Rules to follow for every single problem, no exceptions:

1. **Give yourself 25–30 minutes minimum before looking at ANY hint or solution.** Struggling is where learning happens.
2. **Never copy-paste a solution.** If you must look at one, read it, close it, wait 10 minutes, then re-type it from memory understanding *why* each line exists.
3. **After solving (or seeing the solution), always ask "what pattern was this?"** Write it down. This is the single highest-leverage habit.
4. **Redo problems.** Solving something once means little. Redo it after 3 days, then after 2 weeks, without looking.
5. **Talk out loud (or write) your approach BEFORE coding.** Interviewers and your future self both need this skill.

---

1. The 15 Patterns That Cover ~90% of LeetCode

Learn these in order. Each pattern = a mental template you apply, not a memorized solution.

#	Pattern	Signal words / clues	Starter problems
1	Two Pointers	sorted array, pair sum, palindrome	Two Sum II, Valid Palindrome, Container With Most Water
2	Sliding Window	subarray/substring, "longest/shortest/contains"	Longest Substring Without Repeating Chars, Max Sum Subarray Size K
3	Fast & Slow Pointers	linked list, cycle detection	Linked List Cycle, Middle of Linked List
4	Merge Intervals	overlapping ranges, scheduling	Merge Intervals, Insert Interval
5	Binary Search (+ on answer)	sorted, "find min/max satisfying condition"	Search in Rotated Array, Koko Eating Bananas
6	Hashing / Prefix Sum	count pairs, subarray sum equals K	Two Sum, Subarray Sum Equals K
7	Stack (monotonic)	next greater/smaller, matching brackets	Valid Parentheses, Daily Temperatures
8	Tree DFS/BFS	tree traversal, level order, path sum	Binary Tree Level Order, Path Sum
9	Graph BFS/DFS	grid, islands, shortest path unweighted	Number of Islands, Course Schedule
10	Backtracking	"all combinations/permutations", constraints	Subsets, Permutations, N-Queens
11	Dynamic Programming (1D)	"min/max ways", overlapping subproblems	Climbing Stairs, House Robber
12	Dynamic Programming (2D)	grids, two strings	Longest Common Subsequence, Edit Distance
13	Heap / Priority Queue	"top K", "kth largest", merge K lists	Kth Largest Element, Merge K Sorted Lists
14	Union-Find	connected components, cycles in undirected graph	Number of Provinces, Redundant Connection

#	Pattern	Signal words / clues	Starter problems
15	Greedy	local optimal choice, intervals, jump games	Jump Game, Gas Station

2. The Study Plan (12 weeks, ~1–1.5 hrs/day)

Phase 1 — Foundations (Weeks 1–4): Arrays, Strings, Two Pointers, Sliding Window, Hashing, Stack

- 4–5 problems/week per pattern, mix of Easy → Medium.
- Goal: instantly recognize these 5 patterns from the problem statement.

Phase 2 — Linear Structures (Weeks 5–7): Linked Lists, Trees, BFS/DFS, Graphs

- Draw every tree/graph problem on paper before coding.
- Learn both recursive AND iterative traversal for trees.

Phase 3 — Advanced (Weeks 8–10): Backtracking, DP (1D then 2D), Heaps

- DP is the hardest — for every DP problem, first write the brute-force recursion, THEN memoize, THEN convert to tabulation.
- Don't skip the brute force step. It's how you actually understand DP instead of memorizing patterns.

Phase 4 — Polish (Weeks 11–12): Greedy, Union-Find, mixed random problems

- Do timed mock sessions: 2 mediums in 45 minutes, no hints.
- Revisit your weakest pattern from a personal error log (see below).

3. How to Actually Solve a Problem (your per-problem checklist)

1. **Restate the problem in your own words.** If you can't, you don't understand it yet.
2. **Identify constraints** (input size, value ranges) — they hint at required time complexity:
   - $n \leqslant 20$ → brute force / backtracking OK
   - $n \leqslant 1000$ → $O(n^2)$ OK
   - $n \leqslant 10^5$–10^6 → need $O(n \log n)$ or $O(n)$
3. **Work a small example by hand** on paper.
4. **State your approach out loud in plain English** before writing code.
5. **Code it.**
6. **Trace through your example manually** to verify.
7. **State time & space complexity.**
8. **Only now** compare with the editorial/discussion — never before step 7.

---

4. Track Your Progress: The Error Log (critical, most people skip this)

Keep a spreadsheet or notes doc with these columns: `Date | Problem | Pattern | Solved alone? (Y/N) | Mistake type | Redo date`

Mistake types to tag: *didn't recognize pattern / edge case missed / off-by-one / wrong complexity assumption / syntax error*.

Every Sunday, review this log. If one pattern keeps showing up, that's your next week's focus — not a random new topic.

---

5. Resources (free, high-quality)

- **NeetCode 150 / Blind 75** — the best-curated problem lists, organized exactly by pattern above. Use NeetCode's YouTube explanations only *after* you've struggled for 25+ minutes.
- **"Grokking the Coding Interview"** patterns (many free summaries exist) — pairs well with the table in Section 1.
- **CS50 / MIT 6.006** if a specific data structure (e.g., heaps, tries) feels shaky — go back to fundamentals rather than pattern-matching blindly.
- LeetCode's own **Explore Cards** for structured, free topic-by-topic practice.

---

6. Weekly Targets

- Weeks 1–4: 5 problems/week (mostly Easy, some Medium)
- Weeks 5–8: 6–7 problems/week (mostly Medium)
- Weeks 9–12: 7–8 problems/week (Medium + a few Hard)
- Total by week 12: **~150–180 problems**, which is enough to comfortably recognize any pattern on sight.

Quality over quantity always — 3 problems deeply understood beats 15 solved by peeking at hints.

---

7. Signs You're Actually Mastering This (not just grinding)

- You can categorize a brand-new problem's pattern within 2 minutes of reading it.
- You can explain your solution's time/space complexity without hesitation.
- You can solve variations of a problem you've done before, even if the wording changes completely.
- You start predicting the next test case an interviewer might throw at you (empty input, all duplicates, negative numbers, single element).

Good luck — this is a marathon, not a sprint. Consistency over 12 weeks beats a burnout week of 10 hours/day.